

. DOCUMENTATION OF THE APPLICATION

1. INTRODUCTION

The Student Assistant Application System is a mobile application developed for the Information Technology Department to simplify and manage the Student Assistant application process. The system was developed using Flutter and Supabase and follows the MVVM architectural pattern with Provider state management.

Traditionally, Student Assistant applications were processed manually using paper forms and email communication. This process was time-consuming, difficult to manage, and inefficient when handling large numbers of applicants. The developed system digitises the entire process by allowing students to submit applications electronically while enabling administrators to review and manage applications efficiently.

The application uses Supabase as the backend service for authentication, database storage, and file storage. The project was developed collaboratively using GitHub for version control

SYSTEM OVERVIEW

The Student Assistant Application System provides two main user roles:

2.1 Student Users

Student users are able to:

- Register and login securely
- Submit Student Assistant applications
- Apply for a maximum of two modules
- Upload supporting documents
- View submitted applications
- View application status
- Edit applications while pending
- Delete applications while pending

2.2 Administrative Users

Administrative users are able to:

- Login securely
- View all submitted applications

- Review student details
- Review uploaded supporting documents
- Approve or reject applications
- Update application statuses
- Delete invalid applications
- Filter applications for easier review

The system ensures that users can only access functionality relevant to their assigned roles:

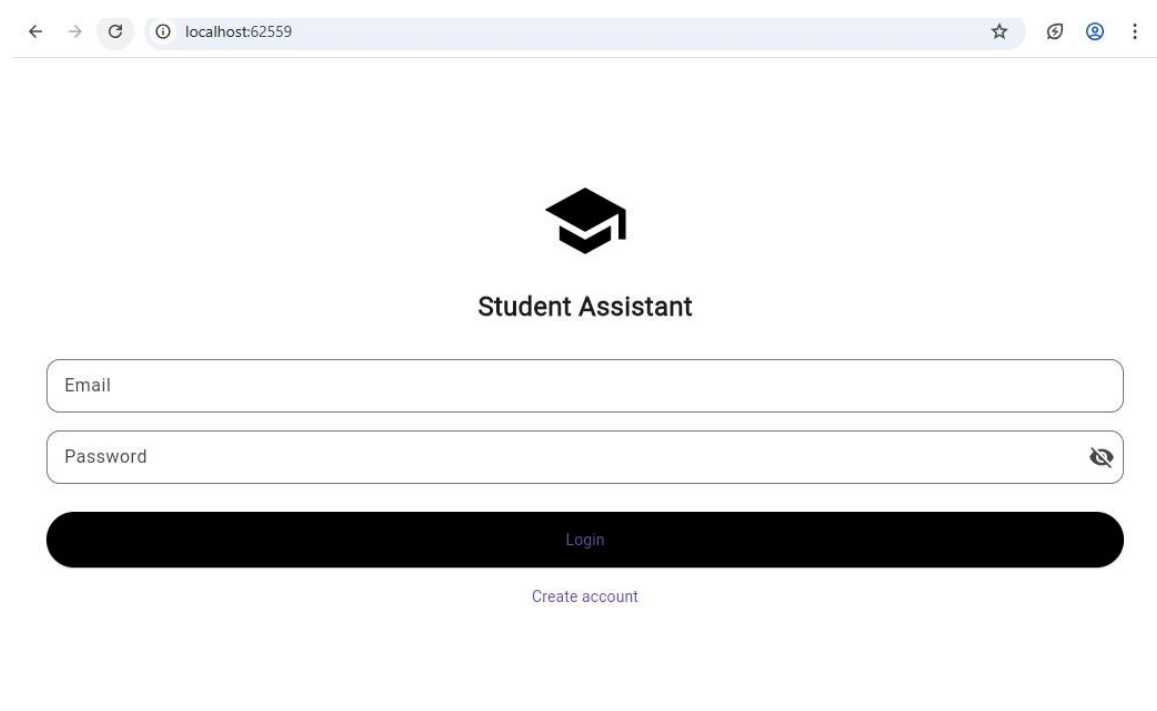
3. SYSTEM OBJECTIVES

The objectives of the system are:

- To provide a secure online platform for Student Assistant applications
- To simplify the application submission process
- To reduce manual paperwork and administrative workload
- To allow administrators to manage applications efficiently
- To implement secure authentication and data storage
- To demonstrate the use of Flutter, MVVM, Provider, routing, validation, and Supabase integration

APPLICATION – SCREEN

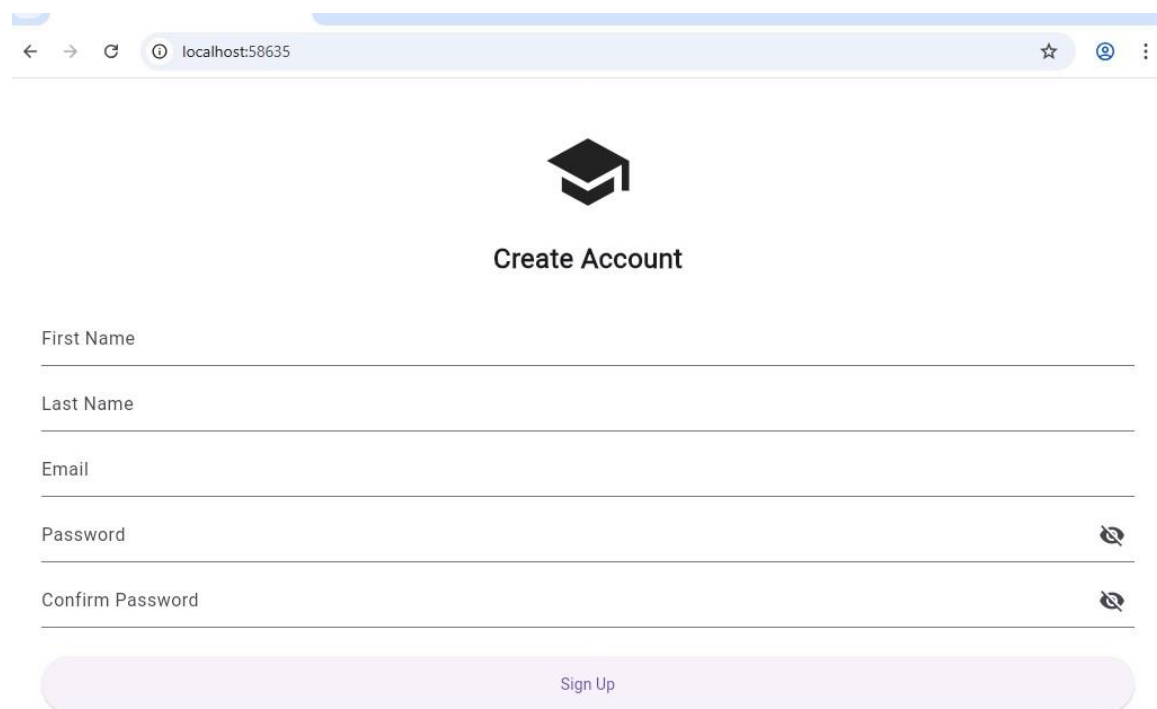
Login Screen



The screenshot shows a web browser window with the address bar displaying 'localhost:62559'. The login screen features a black graduation cap icon at the top center, followed by the text 'Student Assistant'. Below this, there are two input fields: 'Email' and 'Password'. The 'Password' field includes a toggle icon for visibility. A large black 'Login' button is positioned below the input fields, and a smaller 'Create account' link is located at the bottom center.

The Login screen is the system's secure entry point. It allows registered users to access their accounts using an email and password. The email field validates format correctness, while the password field supports secure masked entry and visibility toggle. The Login button authenticates credentials against stored user records. If successful, the system redirects users to the appropriate dashboard depending on role (student or admin). Invalid login attempts trigger validation errors. The Create Account link redirects new users to the registration page.

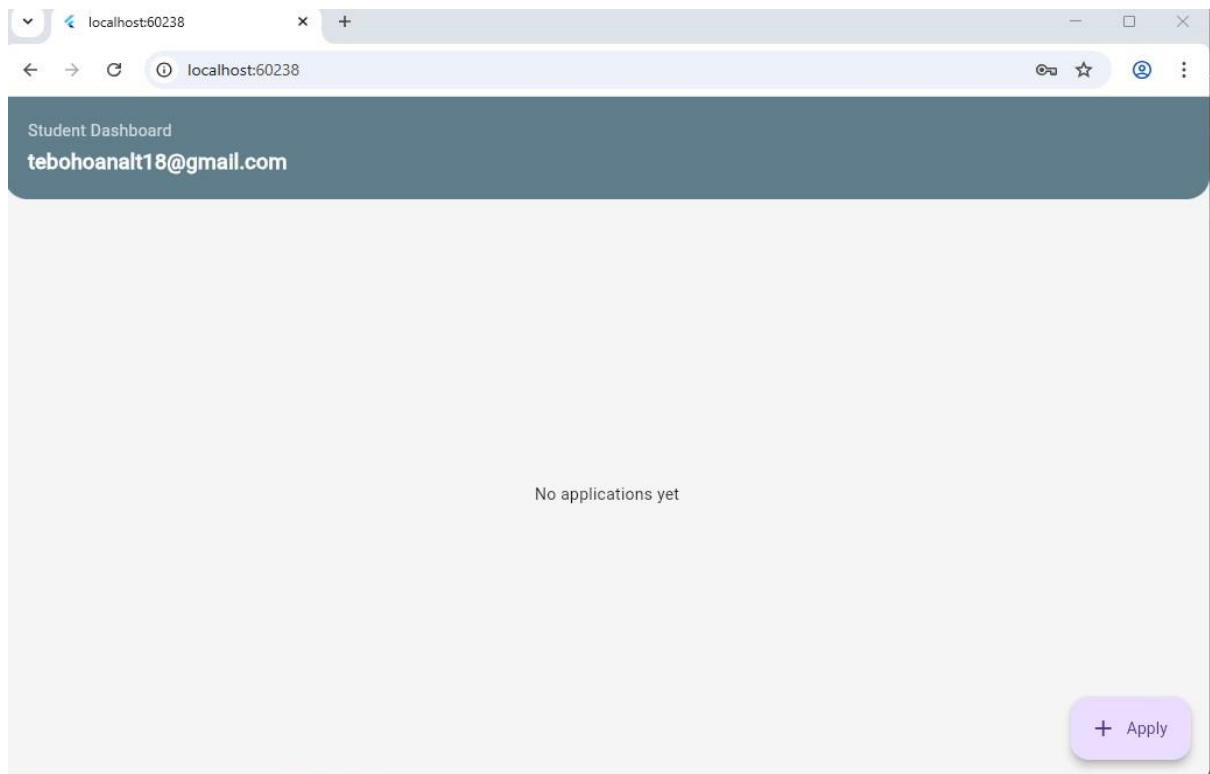
Register Screen



The screenshot shows a web browser window with the address bar displaying 'localhost:58635'. The page features a dark blue header with a white graduation cap icon and the text 'Create Account'. Below the header, there are five input fields: 'First Name', 'Last Name', 'Email', 'Password', and 'Confirm Password'. The 'Password' and 'Confirm Password' fields have a small eye icon to the right, indicating a visibility toggle. At the bottom, there is a light blue rounded button labeled 'Sign Up'.

The Register screen enables new users to create accounts. It collects first name, last name, email, password, and password confirmation. Password confirmation ensures account security through matching validation. The password visibility toggle improves usability. On submission, the system checks field completeness, validates email structure, ensures password strength, and creates a new account if all requirements are met. Once successful, the user can log in immediately.

Student Dashboard



The Student Dashboard is the student's main workspace after login. It displays the logged-in email address for identity confirmation. If no applications exist, the dashboard displays an empty-state message. The floating Apply button initiates a new application process and redirects to the Application Form screen. Once applications exist, this screen would display application records and statuses for tracking progress.

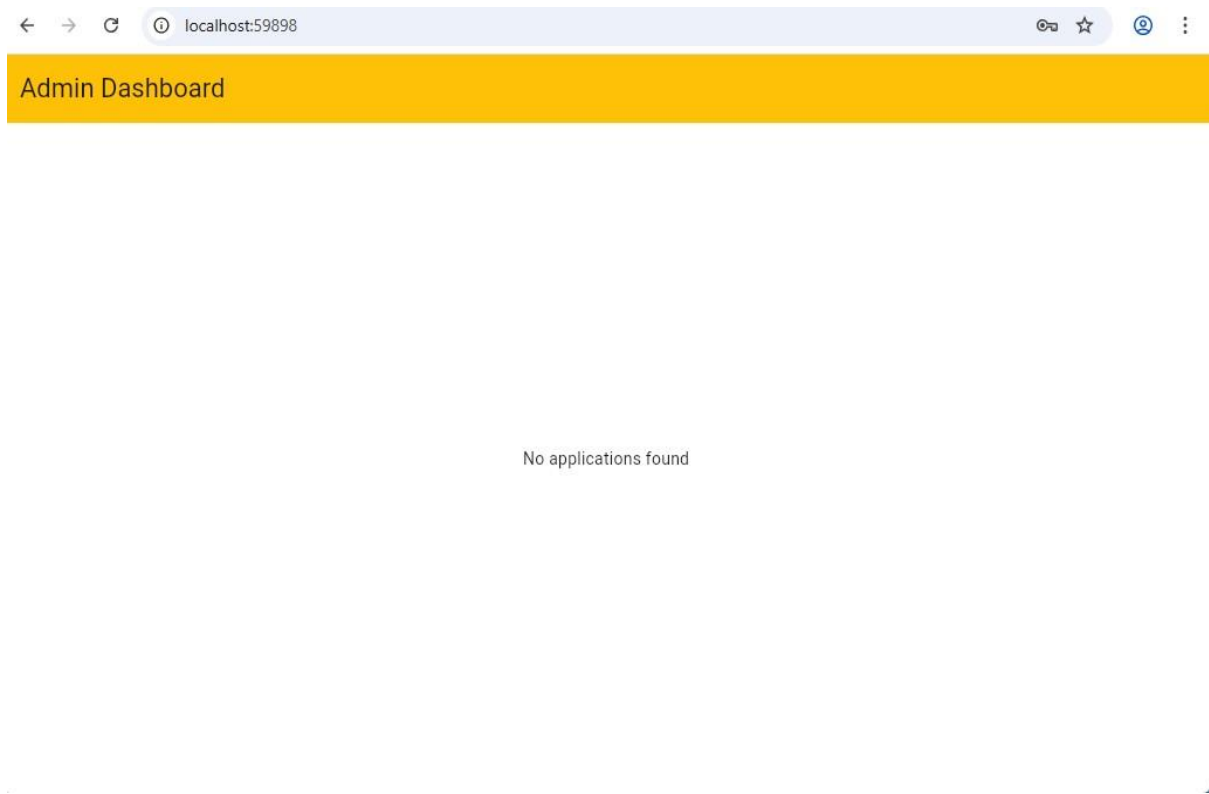
Application Form

The screenshot shows a web browser window with the address bar displaying 'localhost:60238'. The page title is 'Application Form'. The form contains the following elements:

- A header bar with a back arrow and the title 'Application Form'.
- A section labeled 'Module 1' with a text input field.
- A section labeled 'Module 2 (Optional)' with a text input field.
- A confirmation statement 'I confirm eligibility' followed by an unchecked checkbox.
- An 'Upload CV' button, which is currently disabled.
- A text label 'No file selected' below the upload button.
- A large 'Submit' button at the bottom of the form.

The Application Form allows students to submit academic applications. It contains module selection fields for choosing required courses, an optional second module field, an eligibility confirmation checkbox, and a CV upload feature. The file selector attaches supporting documents. The Submit button validates all mandatory fields before sending data to the backend for storage and review. Successful submission updates the dashboard with the application record.

Admin Dashboard



The Admin Dashboard is the administrator control panel used for managing submitted applications. It displays all received applications for review and processing. If no applications exist, the system shows a placeholder message. Once applications are submitted, this screen would list applicants and allow actions such as viewing application details, verifying eligibility, approving, rejecting, or monitoring application status

Technical Explanation

MVVM Architecture

MVVM is a software architectural pattern that helps organize code into three distinct layers, which are the **Model**, **View** and **ViewModel**. It separates UI from the business logic. It improves the app's maintainability and teamwork. MVVM also provides a cleaner project structure.

Model Layer

It represents application data, examples: ApplicationModel, UserModel, and business rules. This layer stored fields such as modules and status. It is connected to the Supabase database. It does not know about the UI, it is mainly about the data.

View Layer

This layer contains Flutter UI screens. It displays data to users.

Examples: LoginScreen and AdminDashboardScreen.

There is no business logic inside views.

ViewModel Layer

It is the middle layer that connects views and models. It holds the application logic such as fetching data, validating input, updating state.

Examples: AuthViewModel and ApplicationViewModel.

Handles authentication and CRUD operations.

Benefits of MVVM

Separation of concerns → UI, logic, and data are independent.

Testability → You can test ViewModels without worrying about UI.

Scalability → Adding new features won't break existing code.

Maintainability → Easier to debug and extend.

Provider State Management

It allows us to share data (state) across our app and automatically update the UI when that data changes. It uses ChangeNotifier for updates. It also improves performance and code organisation.

Why Provider is Powerful

Clean architecture → It fits perfectly with MVVM (ViewModel + Provider).

Scalable → Works for small apps and large apps with multiple roles (students, admins, etc.)

Routing and Navigation

Flutter Navigator handles navigation. Users redirected based on roles. Students open Student Home. Admins open Admin Dashboard.

Additional navigation: open application form, return to home screen and logout to login screen.

Navigation Flow

Login Screen -> Student Home

Login Screen -> Admin Dashboard

Student Home -> Application Form

Admin Dashboard -> Review Applications

Form Handling

Form handling is how you collect, validate, and process user input in Flutter apps.

Examples in our Student Management System: Login form (email + password). Application submission form (student details, course, status).

TextFormField is used for data capture. Controllers manage form values. Form widget groups related fields.

Validation Rules

Required fields cannot be empty. Maximum of two module selections. Only one application allowed per student. Invalid submissions blocked automatically. Email must contain @. Password must be at least 6 characters. Phone number must be digits only.

Supabase Backend

Supabase provides backend services. It handles Authentication and Database. Stores uploaded documents securely. Supports cloud-based storage.

In our Student Management System:

Profiles table → Stores student/admin info.

Applications table → Stores application submissions and statuses.

Supabase handles persistence, while our ViewModel manages logic.

Supabase Authentication

Users login with email and password.

Authentication secures access.

Role-based access implemented.

Unauthorised access blocked.

CRUD Operations

Create: Submit applications.

Read: View applications.

Update: Edit or approve applications.

Delete: Remove invalid applications.

Create Operation

Students complete the application form.

Data inserted into Supabase.

Documents uploaded to storage.

Application status set to Pending.

In MVVM the ViewModel exposes `createApplication()`. The View calls it when the student taps "Submit".

Read Operation

Students view their own applications.

Admins view all applications.

Application statuses displayed dynamically.

Queries retrieve records from Supabase.

In MVVM: The ViewModel fetches data and stores it in a list. The Provider notifies the View to rebuild.

Update Operation

Students edit pending applications.

Admins approve or reject applications.

Database records updated instantly.

Provider refreshes UI automatically.

In MVVM: The ViewModel handles the update logic. The View triggers it when the admin clicks "Approve".

Delete Operation

Students delete pending applications.

Admins remove invalid applications.

Confirmation prevents accidental deletion.

Supabase removes records permanently.

In MVVM: The ViewModel exposes `deleteApplication()`. The View calls it when the admin taps "Delete".

Group Contribution

Names and student numbers of all group members, Clear description of each group member's contribution

Sebariki Refiloe Tshegofatso Choane **221008431**: Authentication Screen (Login-Read Operation) and Home Screen (Access Functionality to submit or manage applications)

Selloane Alinah Leeuw **221003714** : Home screen(view submitted applications, view the current status of each application)

Moelo Evodia Kwetle **221013252** : Application Detail Screen(Read/Delete Operation)

Teboho Mohlomi **220044173**: Backend part (SUPABASE Account) , Student Assistant Application form(Create/ Update Operation) ,Administration Dashboard(Read/Update/Delete operations) and Administration designs

Maipato Mokhele **221027626** : Documentation(Group contribution) Adding everything on documentation

Tshepiso Muriel Choeu **223043312**: Documentation (Technical Explanation)

Lerato Makhetha **221019628** : Documentation (Application Screens)

Ntando Ndlovu **22303645**: Documentation (System Overview)